other hand, as shown in table 3.8, the same word appearing twice or more in the title field (with fewer words) says that the book is about the Java programming language.

**Table 3.8 Comparing different fields to gather similarity**

| Word | Field length | Frequency | Relevant? |
|---|---|---|---|
| *Java* | Synopsis field with a length of 100 words | 1 | No |
| *Java* | Title field with a length of 5 words | 2 | Yes |

In most cases, the BM25 algorithm is adequate. However, if we need to swap BM25 with another algorithm, we can do so by configuring it using the indexing APIs. Let's go over the mechanics of configuring the algorithm as needed.

### CONFIGURING SIMILARITY ALGORITHMS

Elasticsearch allows us to plug in other similarity algorithms if the default BM25 doesn't suit our requirements. Two similarity algorithms are provided out of the box without further customization: `BM25` and `boolean`. We can set the similarity algorithm for individual fields when we create the schema definitions using index settings APIs, as shown in figure 3.20.

> **NOTE** Working with similarity algorithms is an advanced topic. While I advise you to read this section, you can skip it and revisit it when you wish to know more.

```
PUT index_with_different_similarities
{
  "mappings": {
    "properties": {
      "title":{
        "type": "text",
        "similarity": "BM25"
      },
      "author":{
        "type": "text",
        "similarity": "boolean"
      }
    }
  }
}
```

**Creates an index with two fields**

**The `title` field is defined with a `BM25` (default) similarity explicitly.**

**The `author` field is defined with a `boolean` similarity function.**

**Figure 3.20 Setting fields with different similarity functions**

In the figure, an index `index_with_different_similarities` is being developed with a schema that has two fields: `title` and `author`. The important point is the specifica-